

DevSecOps Security Assessment of OWASP WebGoat

Threat Modeling · SAST / SCA / DAST · Manual Code Review · CI/CD Security Pipeline

Project 1 (E15) — Secure Software Development

Instructor: Hoang Viet Dung

Ha Minh Phong — 202416817

Nguyen Dinh Khanh Nguyen — 202416816

Repository: <https://github.com/mehatecodethings/WebGoat>

Table of Contents

1. Introduction	4
1.1 Target and environment	4
2. Threat Model (STRIDE)	5
2.1 Scope and system overview	5
2.2 Data flow and trust boundaries	5
2.3 Assets to protect	5
2.4 Threat analysis by STRIDE	5
2.5 Mapping the discovered vulnerabilities to STRIDE	6
Classification rationale	6
2.6 Mitigations (summary)	6
3. Manual Code Review & Proof of Concept	8
3.1 SQL Injection	8
Vulnerable code (registration / user-existence check)	8
Why it is vulnerable	8
Proof of concept — authentication bypass	8
Impact	9
Fix — parameterized query (PreparedStatement)	9
Why the fix works	9
3.2 Stored (Persistent) Cross-Site Scripting	11
Vulnerable code (comment storage and retrieval)	11
Why it is vulnerable	11
Proof of concept — executing an injected script	11
Impact	11
Fix — encode / sanitize on output	12
Why the fix works	12
3.3 Path Traversal (Zip Slip)	13
Vulnerable code (archive extraction)	13
Why it is vulnerable	13
Proof of concept — overwriting a file outside the upload directory	13
Impact	14
Fix — validate the resolved path before writing	14
Why the fix works	14
4. SAST Findings & True / False-Positive Analysis	15
4.1 Semgrep	15

4.2 SonarQube Cloud	15
4.3 True vs false positives	16
4.4 Why manual review is still required	16
5. DevSecOps Pipeline.....	17
5.1 Stages and tool choices.....	17
5.2 Quality Gate and blocked deployment	17
5.3 Engineering notes (gotchas resolved).....	18
5.4 One-time setup (secrets)	18
5.5 Closing the loop: fix → re-scan → pass.....	18

1. Introduction

This report documents a security assessment of OWASP WebGoat, a deliberately vulnerable Java (Spring Boot) web application maintained by OWASP for security learning and testing. WebGoat was chosen because it is a full, running web application, which lets the assessment exercise the complete range of techniques required by the project: static analysis of the source, analysis of its third-party dependencies, and dynamic testing of the deployed application.

The assessment is organized into four areas, one per chapter. The threat model applies STRIDE to the application and its trust boundaries. The manual code review presents three confirmed vulnerabilities — SQL Injection, Stored Cross-Site Scripting, and Path Traversal (Zip Slip) — each with the exact vulnerable code, a working proof of concept that explains the underlying mechanism, the impact, and a fix with a detailed explanation of why it removes the flaw. The SAST chapter reviews the automated findings from Semgrep and SonarQube Cloud and separates true positives from false positives. Finally, the DevSecOps pipeline chapter describes a GitHub Actions workflow that runs SAST, SCA, and DAST on every change behind a quality gate that fails the build on high or critical findings.

Together these chapters show both how the vulnerabilities arise in the code and how an automated, secure-by-default development process detects and blocks them before release.

1.1 Target and environment

Item	Detail
Target application	OWASP WebGoat (release 2025.3), Java 21 / Spring Boot
Source repository	github.com/mehatecodethings/WebGoat (fork)
Runtime	Docker container, http://localhost:8080/WebGoat
Embedded database	HSQLDB / in-memory, stores accounts and lesson data
CI/CD	GitHub Actions — .github/workflows/devsecops.yml
SAST tools	Semgrep (p/java) + SonarQube Cloud
SCA tool	Snyk (Maven dependencies)
DAST tool	OWASP ZAP baseline scan

2. Threat Model (STRIDE)

2.1 Scope and system overview

WebGoat is a Java web application (Spring Boot) that deliberately contains vulnerabilities for security learning. The scope of this threat model is WebGoat deployed locally (localhost:8080), comprising the web tier (controllers / REST endpoints and the UI), the lesson logic, and an embedded in-memory database that stores user accounts and lesson data.

Goal: identify threats across the six STRIDE categories on each component and trust boundary, then map the three vulnerabilities found during manual code review (SQL Injection, Stored XSS, Path Traversal) onto the correct category and propose mitigations.

2.2 Data flow and trust boundaries

The system is modeled with three elements: an external entity — the user / browser (including the attacker); a process — the WebGoat application; and a data store — the embedded database plus the server file system. A trust boundary is a place where the trust level of data changes and a control is therefore required.

Boundary	Description
TB1	Between the browser (untrusted) and the application — all client-supplied data must be treated as untrusted.
TB2	Between the application and the database / file system — statements must be parameterized and file paths validated, not built by concatenation.
TB3	Between the data and sessions of different users — user A's data must not be exposed to or manipulated by user B.
TB4	Between the unauthenticated and the authenticated zones.

2.3 Assets to protect

- User credentials (username, password) and the login session.
- Users' personal / profile data (confidentiality).
- Integrity of the data stored in the database and on the file system.
- Availability of the application.

2.4 Threat analysis by STRIDE

The table lists a representative threat for each STRIDE category, tied to the corresponding component or trust boundary.

STRIDE category	Typical threat on WebGoat	Component · boundary
Spoofing	Logging in as another user via a weak password, missing lockout, or a hijacked session.	Login flow; TB1, TB4

STRIDE category	Typical threat on WebGoat	Component · boundary
Tampering	Malicious input alters the DB query, injects script into stored data, or writes files outside the intended folder via a crafted upload.	Input crossing TB1 → TB2
Repudiation	Missing logs / audit means illicit actions cannot be traced.	Application (no logging)
Information Disclosure	Accessing or extracting unauthorized data: other tables, other users' profiles.	TB2, TB3
Denial of Service	Large payloads or heavy queries exhaust application and DB resources.	Application + DB
Elevation of Privilege	Acting as another user or admin beyond one's rights; script running in the victim's session.	TB3, TB4

2.5 Mapping the discovered vulnerabilities to STRIDE

The three vulnerabilities confirmed during code review are concrete instances of the threats above.

Vulnerability (file)	Primary STRIDE category	Boundary violated	Risk
SQL Injection (SqlInjectionChallenge.java)	Tampering + Information Disclosure	TB1 → TB2	High / Critical
Stored XSS (StoredXssComments.java)	Tampering + Elevation of Privilege	TB1, TB3	High
Path Traversal / Zip Slip (ProfileZipSlip.java)	Tampering (arbitrary file write)	TB1 → TB2 (file system)	High / Critical

Classification rationale

- **SQL Injection:** untrusted client input (TB1) is concatenated straight into the SQL statement, crossing the App–DB boundary (TB2). The result is both a changed query meaning (Tampering) and reading data beyond what is allowed (Information Disclosure).
- **Stored XSS:** user input (TB1) is stored and later rendered as code in another user's browser (TB3). This changes the page content (Tampering) and runs code inside the victim's session to act on their behalf (Elevation of Privilege).
- **Path Traversal / Zip Slip:** the application extracts an uploaded archive (TB1) and writes each entry using its name as-is. A crafted entry name containing "../" escapes the intended folder and writes elsewhere on the server (TB2) — an arbitrary file write (Tampering) that, by overwriting application or config files, can lead to code execution (Elevation of Privilege).

2.6 Mitigations (summary)

Threat	Mitigation
SQL Injection	Use PreparedStatement / parameterized queries; grant the DB account least privilege; validate input.
Stored XSS	Context-aware output encoding; sanitize input; set a Content-Security-Policy header; mark the session cookie HttpOnly.

Threat	Mitigation
Path Traversal / Zip Slip	Canonicalize the resolved path and verify it stays inside the target directory before writing; reject entries containing "..".
Spoofing	Strong passwords, account lockout after repeated failures, safe session management.
Repudiation	Enable logging / audit for sensitive actions.
Denial of Service	Limit input size, paginate results, cap resources a query may consume.

Detailed exploitation (PoC) and fixes for the three main vulnerabilities are given in the next chapter.

3. Manual Code Review & Proof of Concept

The source was reviewed manually, focusing on the points where untrusted user input reaches a sensitive sink: SQL queries (injection), HTML output rendered to other users (cross-site scripting), and the handling of uploaded archives (path traversal). Three vulnerabilities were confirmed, and each is documented below with the exact vulnerable code, a working exploit explained at the mechanism level, the impact, and a fix with reasoning. All exploits were reproduced against a local WebGoat instance (<http://localhost:8080/WebGoat>) after registering an account.

3.1 SQL Injection

File: `src/main/java/org/owasp/webgoat/lessons/sqlinjection/advanced/SqlInjectionChallenge.java`

Vulnerable code (registration / user-existence check)

```
String checkUserQuery =
    "select userid from sql_challenge_users where userid = '"
    + username + "'";
Statement statement = connection.createStatement();
ResultSet resultSet = statement.executeQuery(checkUserQuery);
```

Why it is vulnerable

The value `username` comes directly from the request and is concatenated into the SQL string with the `+` operator, then executed through a plain `Statement`. The database receives one finished string and cannot distinguish which characters were the developer's command and which were user data. A single quote (`'`) typed by the user is therefore parsed as SQL syntax rather than as text — the classic string-concatenation injection. The sibling login endpoint in the same package (`SqlInjectionChallengeLogin.java`) does this correctly with a `PreparedStatement`, which makes the contrast a clean teaching example: identical intent, one safe and one unsafe, differing only in how the query is built.

Proof of concept — authentication bypass

The advanced lesson asks the attacker to log in as the user "tom" without knowing his password. Because the underlying challenge family builds its lookup by concatenation, a tautology payload makes the `WHERE` clause always true and the password check collapses:

- **Username field:** tom
- **Password field:** Tom' OR 1=1--

Conceptually the server evaluates a statement of the form:

```
... WHERE userid = 'tom' AND password = 'Tom' OR 1=1--
```

The injected single quote closes the password literal early; `OR 1=1` is always true; and the double dash `--` comments out the remainder of the statement (including the original closing quote and any further conditions). The condition therefore evaluates to true regardless of the real password, and the application authenticates the session as tom.

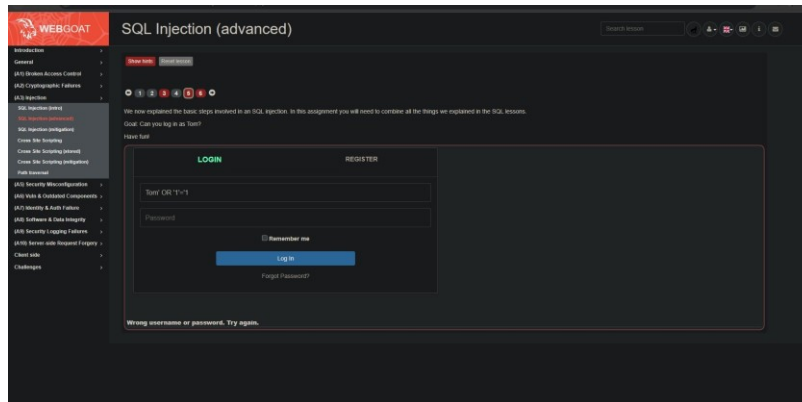


Figure 3.1 — The login form with the tautology payload entered in the password field.

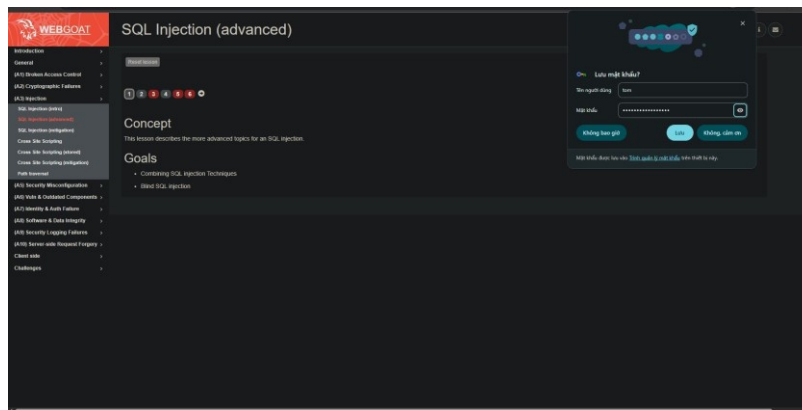


Figure 3.2 — WebGoat authenticates the session as tom without the correct password: SQL injection confirmed.

Impact

An attacker bypasses authentication and reads or acts on records they are not authorized to see. With richer payloads (UNION SELECT, stacked queries) the same flaw allows reading other tables such as the accounts table, or modifying and deleting data. In the threat model this is Tampering and Information Disclosure across TB1 → TB2.

Fix — parameterized query (PreparedStatement)

```
PreparedStatement checkUserStatement = connection.prepareStatement(
    "select userid from sql_challenge_users where userid = ?");
checkUserStatement.setString(1, username);
ResultSet resultSet = checkUserStatement.executeQuery();
```

Why the fix works

With a PreparedStatement the SQL text containing the ? placeholder is sent to the database and compiled before any user value is supplied. The call to setString then binds username as a pure data parameter that is never re-parsed as part of the command. Whatever the user types — including quotes, OR 1=1, or comment markers — is treated as a single literal value to compare against the userid column, so it cannot change the structure of the query. The injection is neutralized at the boundary where data becomes a command. As defense in depth, the database account should run with least privilege and the inputs should still be length- and format-validated.

3.2 Stored (Persistent) Cross-Site Scripting

File: src/main/java/org/owasp/webgoat/lessons/xss/stored/StoredXssComments.java

Vulnerable code (comment storage and retrieval)

```
@PostMapping("/CrossSiteScriptingStored/stored-xss")
public ActionResult createNewComment(
    @RequestBody String commentStr, @CurrentUsername String username) {
    Comment comment = parseJson(commentStr);
    ...
    comments.add(comment);          // text stored verbatim
    userComments.put(username, comments);
    ...
}

// retrieveComments() later returns the stored text unchanged,
// which the front end renders into the page as HTML.
```

Why it is vulnerable

A submitted comment is stored exactly as received and is later returned by `retrieveComments()` and rendered into the HTML page shown to every viewer, with no HTML-encoding or sanitization at any point. Because the browser receives the stored characters as part of the page markup, a comment containing a `<script>` element is parsed and executed as code rather than displayed as text. The payload is persistent: it runs for every user who later loads the comments — including an administrator — which is what makes stored XSS more dangerous than the reflected variety.

Proof of concept — executing an injected script

The lesson provides a hook, `webgoat.customjs.phoneHome()`, whose successful invocation proves arbitrary script execution. The following comment is submitted in the comment box:

```
<script>webgoat.customjs.phoneHome()</script>
```

When the comment list re-renders, the injected script runs in the page context and calls `phoneHome()`. Opening the browser DevTools console shows the function executing and returning a one-time response value ("phoneHome Response is ..."). Submitting that returned value to the lesson's follow-up field confirms that our injected JavaScript — not a built-in lesson script — produced it:

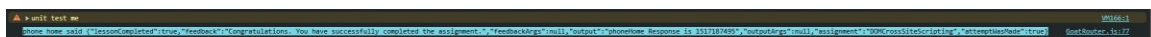


Figure 3.3 — DevTools console: the injected script executes and `phoneHome` returns its response value.

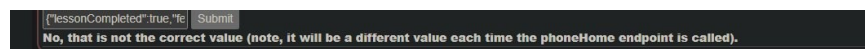


Figure 3.4 — Submitting the returned `phoneHome` value completes the assignment: stored XSS confirmed.

Impact

The injected script runs with the victim's session, in their browser, every time the poisoned page loads. A real attacker could steal the session cookie, perform actions as the victim, deface the page, or redirect users to a phishing site. Because the payload is stored server-side and served to all viewers, a single comment can compromise many users, including privileged ones. In the threat model this is Tampering and Elevation of Privilege across TB1 and TB3.

Fix — encode / sanitize on output

Add the import and HTML-escape the comment text before it is stored (or, equivalently, at the point of rendering):

```
import org.springframework.web.util.HtmlUtils;
...
comment.setUser(username);
comment.setText(HtmlUtils.htmlEscape(comment.getText())); // neutralize markup
comments.add(comment);
```

Why the fix works

`HtmlUtils.htmlEscape()` converts the characters that have meaning in HTML — `<` `>` `"` `&` — into their entity equivalents (`<` `>` `"` `&`). After escaping, the stored value `<script>...</script>` becomes `<script>...</script>`, which the browser renders as the literal text of a comment instead of parsing it as an element. The dangerous payload is therefore displayed, not executed. Encoding at the output context is the canonical XSS defense because it neutralizes the payload exactly where it would otherwise turn from data into code. As additional layers, a Content-Security-Policy header restricts what scripts may run, and marking the session cookie `HttpOnly` prevents script from reading it even if an encoding gap is ever introduced.

3.3 Path Traversal (Zip Slip)

File: src/main/java/org/owasp/webgoat/lessons/pathtraversal/ProfileZipSlip.java

Vulnerable code (archive extraction)

```
ZipFile zip = new ZipFile(uploadedZipFile.toFile());
Enumeration<? extends ZipEntry> entries = zip.entries();
while (entries.hasMoreElements()) {
    ZipEntry e = entries.nextElement();
    File f = new File(tmpZipDirectory.toFile(), e.getName()); // name used as-is
    InputStream is = zip.getInputStream(e);
    Files.copy(is, f.toPath(), StandardCopyOption.REPLACE_EXISTING);
}
```

Why it is vulnerable

The destination of each extracted file is built from `e.getName()` — the entry name stored inside the archive — with no check that the resulting path stays inside the intended extraction directory. The entry name is fully attacker-controlled: a ZIP can legally contain an entry literally named `../../../something`. When such a name is joined to the base directory and resolved by the file system, the `../` sequences walk upward out of the target folder. The code trusts the contents of the archive instead of validating the destination, which is the Zip Slip flaw.

Proof of concept — overwriting a file outside the upload directory

The assignment helpfully states the absolute path of the profile image to overwrite. A malicious archive is built locally with a single entry whose name traverses up and back down to that target path. Using Python (run from a folder containing a throwaway image at `traversal\michaeljordan.jpg`):

```
import zipfile
with zipfile.ZipFile("malicious.zip", "w") as zf:
    zf.write(
        "traversal/michaeljordan.jpg",
        "../../../../../home/webgoat/.webgoat-2025.3/PathTraversal/"
        "michaeljordan/michaeljordan.jpg") # entry name carries the traversal
```

`malicious.zip` is then uploaded through the Zip Slip assignment's file-upload form. On extraction the application honours the traversal path in the entry name and writes the file to the targeted location outside the per-user upload folder. WebGoat detects that the profile image at the protected path changed and reports the assignment solved:

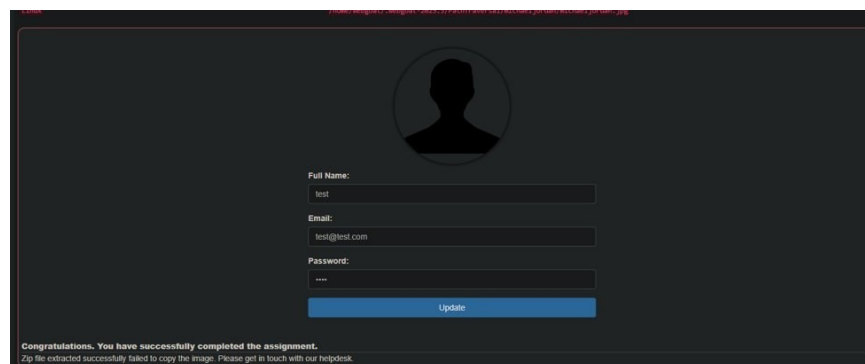


Figure 3.5 — "Congratulations" message after uploading the crafted archive: the entry escaped the upload directory and overwrote a file at the targeted path.

Impact

Arbitrary file write outside the intended directory. Depending on what the server process can write, an attacker can overwrite application or configuration files, replace a script or binary, or plant a web shell, any of which can escalate to remote code execution. In the threat model this is Tampering (arbitrary file write) and, by overwriting executable or configuration files, Elevation of Privilege, crossing TB1 → TB2 (file system).

Fix — validate the resolved path before writing

```
File f = new File(tmpZipDirectory.toFile(), e.getName());
String canonicalDestination = f.getCanonicalPath();
String canonicalTarget = tmpZipDirectory.toFile().getCanonicalPath();
if (!canonicalDestination.startsWith(canonicalTarget)) {
    throw new IOException("Zip Slip detected: " + e.getName());
}
Files.copy(is, f.toPath(), StandardCopyOption.REPLACE_EXISTING);
```

Why the fix works

`getCanonicalPath()` resolves the full, absolute path after collapsing every `.` and `..` segment and following the directory structure — so an entry such as `../../evil.sh` is turned into the actual location it would land in. The guard then checks that this resolved destination still begins with the canonical path of the intended target directory. Any entry that would escape the folder produces a destination that fails the `startsWith` check, and extraction is aborted with an exception before a single byte is written. The control is placed on the destination rather than on the entry name itself, which is important: blacklisting the string `..` can be bypassed with encodings or absolute paths, whereas comparing canonical paths reasons about where the file will truly be written, regardless of how the name was crafted. This is the same principle the lesson's own `ProfileUploadFix` applies when it neutralizes `../../` in the supplied name.

4. SAST Findings & True / False-Positive Analysis

Two static-analysis tools were run over the source as part of the pipeline: Semgrep with the p/java ruleset, and SonarQube Cloud. This chapter summarizes their output, separates genuine issues from noise, and relates the confirmed true positives back to the three vulnerabilities in the manual code review.

4.1 Semgrep

Semgrep scanned 314 Java files with 60 Java rules and produced 17 blocking code findings. The most relevant finding for this assessment is the formatted-SQL-string rule, which flags exactly the concatenation pattern analyzed in section 3.1:

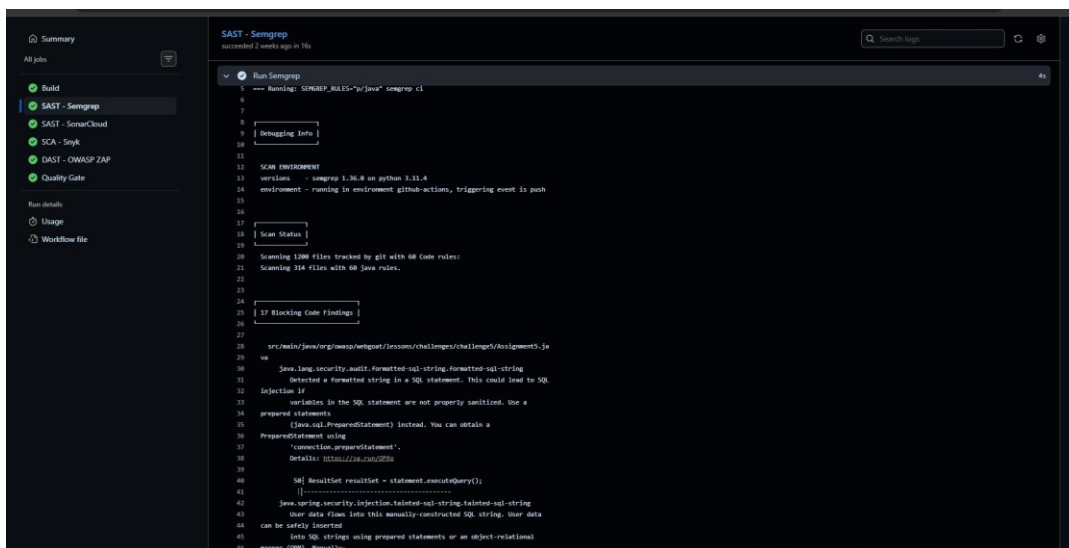


Figure 4.1 — Semgrep run from the pipeline: 17 blocking findings, including the formatted-sql-string rule pointing at a manually-constructed SQL statement.

The rule `java.lang.security.audit.formatted-sql-string` detects a formatted / concatenated string used in a SQL statement and recommends a `PreparedStatement` instead — precisely the SQL-injection true positive confirmed by exploitation. A companion rule, `java.spring.security.injection.tainted-sql-string`, reports that user data flows into a manually-constructed SQL string, corroborating the same issue through taint tracking.

4.2 SonarQube Cloud

SonarQube Cloud analyzed roughly 13k lines of code and reported 97 security issues across blocker, high, medium, and low severities, with the project's quality gate shown as passed for the configured conditions. The dashboard gives the high-level distribution used to prioritize manual review:

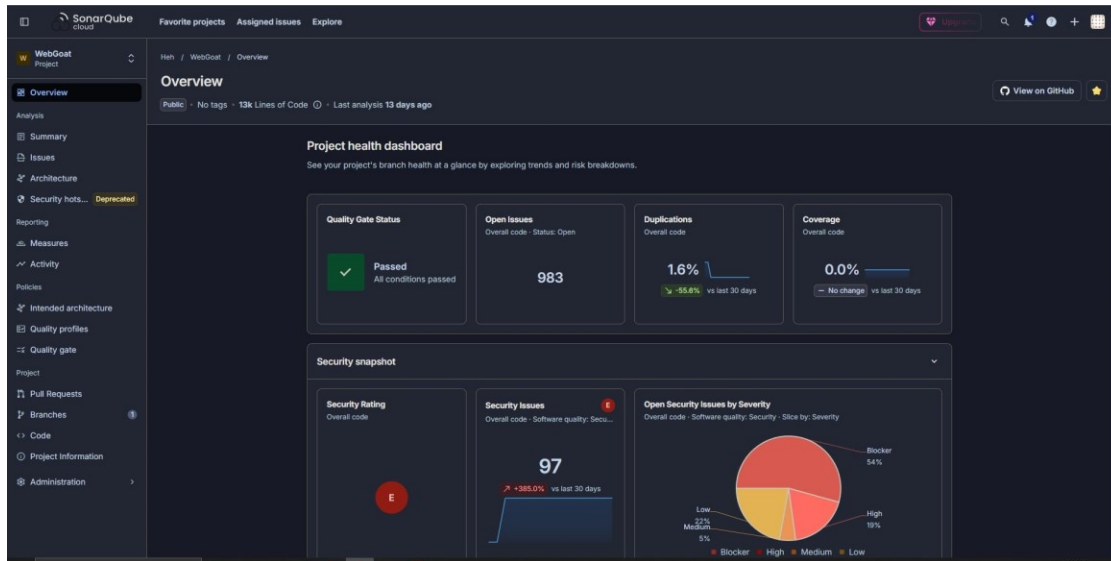


Figure 4.2 — SonarQube Cloud project dashboard: security issue count and severity breakdown (blocker / high / medium / low).

4.3 True vs false positives

Not every machine finding is a real, reachable vulnerability. The table separates representative findings into true and false positives with a short justification.

Finding (rule / type)	Verdict	Reasoning
formatted-sql-string / tainted-sql-string (Semgrep)	True positive	Confirmed by exploitation in 3.1 — user input is concatenated into a query executed via Statement.
Stored XSS / unescaped output (SonarQube hotspot)	True positive	Confirmed by exploitation in 3.2 — comment text is stored and rendered without encoding.
Path / file write from untrusted input (SonarQube)	True positive	Confirmed by exploitation in 3.3 — Zip Slip writes outside the target directory.
SQL-string findings inside other lesson classes	Context-dependent	Many are genuine by design (WebGoat is intentionally vulnerable) but are not in scope for this report's three exploits.
Hard-coded credentials / weak hash in test or seed data	False positive (for risk)	Flagged by pattern but located in lesson seed data, not a deployable secret; no real exposure.
Code-quality / maintainability issues (duplications, smells)	Not security	Valid quality signals but unrelated to exploitable vulnerabilities.

4.4 Why manual review is still required

SAST reliably flags pattern-based flaws such as SQL injection and unescaped output, because these have recognizable code shapes and taint paths. It is much weaker on logic-dependent issues — broken access control, authorization gaps, and some path-traversal cases — where whether the code is vulnerable depends on intent and runtime context the scanner cannot infer. The Zip Slip case illustrates this: the dangerous line looks like ordinary file I/O, and only reasoning about attacker-controlled entry names reveals the flaw. This is exactly why the automated findings were complemented by manual code review and confirmed with proofs of concept.

5. DevSecOps Pipeline

The pipeline is a single GitHub Actions workflow at `.github/workflows/devsecops.yml`. It runs on every push to main and on every pull request, moving through the stages required by the assignment: Code Commit, Build, SAST, SCA, DAST, and a Quality Gate that blocks the build on high / critical findings. All stages pass on the current revision:

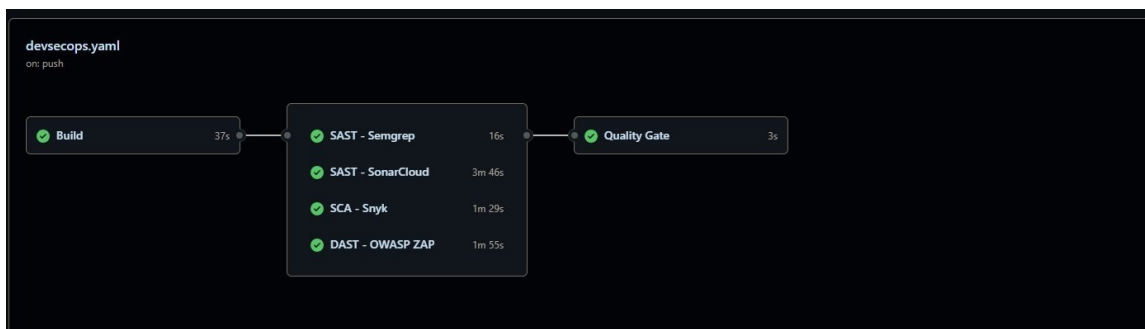


Figure 5.1 — The DevSecOps workflow: Build → (Semgrep, SonarCloud, Snyk, ZAP) → Quality Gate, all green.

5.1 Stages and tool choices

Stage	Tool	What it does	Gate
Code Commit	Actions trigger	Runs the workflow on push to main and on every pull request.	—
Build	Maven (JDK 25)	Compiles WebGoat so later stages have an artifact to analyze.	build error
SAST	Semgrep	Scans source for vulnerable patterns (p/java ruleset).	high finding
SAST	SonarQube Cloud	Deeper static analysis: security issues, hotspots, code quality, managed quality gate.	gate fails
SCA	Snyk	Checks Maven dependencies for known CVEs.	high / critical CVE
DAST	OWASP ZAP	Attacks the running WebGoat for runtime issues (headers, input handling).	ZAP alert
Quality Gate	Gate job	Queries the SonarCloud gate via API; fails the workflow if not OK.	any scan failed

Because WebGoat is a full, running web application, all four scan types apply and none is omitted. SAST and SCA read the code and its dependencies; DAST exercises the deployed application and catches issues that only appear at runtime, such as missing security headers or how input is reflected. Two SAST tools are used because they are complementary: Semgrep applies fast, rule-based pattern matching, while SonarQube Cloud adds broader analysis, security hotspots, and a managed quality gate. For a different target the selection would change — for example a firmware / IoT repository with no running web endpoint would keep SAST and SCA but omit DAST, because there is nothing for ZAP to attack.

5.2 Quality Gate and blocked deployment

The scanners run as reporting steps, and enforcement is handled by a dedicated quality-gate job that runs after all of them. It queries the SonarCloud Quality Gate through the API and fails the workflow if the status is not OK. With branch protection, that failure blocks the merge.

```
quality-gate:
  needs: [sast-semgrep, sast-sonarcloud, sca-snyk, dast-zap]
  steps:
    - name: Check SonarCloud Quality Gate
      run: |
        STATUS=$(curl -s -u ${ secrets.SONAR_TOKEN }: \
          "https://sonarcloud.io/api/qualitygates/project_status?\
projectKey=mehatecodethings_WebGoat" \
          | python3 -c "import sys,json;\
print(json.load(sys.stdin)['projectStatus']['status'])")
        if [ "$STATUS" != "OK" ]; then exit 1; fi
```

Marking the quality-gate job as a required status check in the repository's branch-protection rules makes GitHub refuse to merge a pull request until it passes.

5.3 Engineering notes (gotchas resolved)

- Build originally failed with "release version 25 not supported" — WebGoat's pom.xml requires JDK 25, so both setup-java steps were pinned to Java 25.
- SonarCloud failed with "CI analysis while Automatic Analysis is enabled" — Automatic Analysis was disabled in the SonarCloud project so the CI-driven scan is authoritative.
- The DAST job's container health check killed WebGoat before it finished starting — the service block was replaced with a manual docker run -d plus a wait loop polling /WebGoat/login, which is more robust.
- The quality-gate action looked for .scannerwork/report-task.txt that does not survive across separate jobs — replaced with a direct SonarCloud API call, removing the cross-job file dependency.

5.4 One-time setup (secrets)

- SONAR_TOKEN — a SonarCloud project token, added as a repository secret; the organization and project key (mehatecodethings_WebGoat) are set in the workflow.
- SNYK_TOKEN — a Snyk API token added as a repository secret; Snyk reads the Maven pom.xml automatically.

5.5 Closing the loop: fix → re-scan → pass

After confirming each vulnerability, the corresponding fix from chapter 3 (PreparedStatement for SQLi, HtmlUtils.htmlEscape for stored XSS, canonical-path validation for Zip Slip) is committed on the repository. Pushing the fixes triggers the pipeline again; the SAST stages no longer flag the corresponding patterns in the reviewed files, and the quality gate stays green — demonstrating the full secure-development cycle of detect, exploit, remediate, and verify.